SQL Practice Assignment 3: Arithmetic Expressions & Aliases

Topic: Arithmetic Operators (+, -, *, /), Column Aliases (AS), and Expressions in Clauses

Role: Junior Data Analyst at "TechTrendz Retail"

Part 1: Context & Setup

Context: The Finance Department needs projections for the upcoming Quarter. They don't just want raw data; they want you to calculate potential revenue, simulate discount scenarios, and estimate profit margins.

Data Source: Use the products table you built and updated in Assignments 1 & 2.

Key Concept - Column Aliases:

When you perform a calculation (like price * 2), SQL will often name the column something ugly like ?column? or price * 2. Use the AS keyword to give it a professional name. ● *Example:* SELECT price * 2 AS double_price FROM products;

Part 2: Financial Analysis Questions

Write SQL queries to solve the following business requests.

A. Calculations in the SELECT List

1. **Total Inventory Value:** Management wants to know how much capital is tied up in stock for each item.
   - Select product_name.
   - Create a new column calculated as price multiplied by stock_quantity.
   - **Alias** this new column as inventory_value.

   -- Calculate total inventory value for each product
   -- inventory_value = price * stock_quantity

SELECT product_name,
price * stock_quantity AS inventory_value
FROM products;

```
-- Calculate total inventory value for each product
-- inventory_value = price * stock_quantity

SELECT product_name,
       price * stock_quantity AS inventory_value
FROM products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	inventory_value
Wireless Noise-Canceling Headphones	14999.50
Ergonomic Office Chair	3000.00
Gaming Laptop Pro	12000.00
Stainless Steel Water Bottle	2500.00
Bluetooth Speaker Mini	6898.50
Running Shoes - Size 10	2699.70
Leather Wallet	2700.00
4K Smart TV 55 inch	9000.00
Yoga Mat	4000.00
Coffee Maker	3020.00
Smart Watch Series 5	6250.00
Cotton T-Shirt	4500.00
Gaming Mouse	0.00
Desk Lamp	0.00
Ceramic Mug Set	2000.00
Smart Glasses Prototype	7500.00
Mystery Grab Bag	750.00

2. **Holiday Sale Simulation (20% Off):** The marketing team wants to see what our prices would look like during a 20% off sale.
   - Select product_name and the current price.
   - Create a new column calculated as price multiplied by 0.80 (which represents 80% of the original price).
   - **Alias** this new column as discounted_price.

-- Calculate discounted price after 20% discount
-- discounted_price = price * 0.80
SELECT product_name,

```
    price,
    price * 0.80 AS discounted_price
FROM products;
```

```
-- Calculate discounted price after 20% discount
-- discounted_price = price * 0.80
SELECT product_name,
       price,
       price * 0.80 AS discounted_price
FROM products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	price	discounted_price
Wireless Noise-Canceling Headphones	299.99	239.9920
Ergonomic Office Chair	150.00	120.0000
Gaming Laptop Pro	1200.00	960.0000
Stainless Steel Water Bottle	25.00	20.0000
Bluetooth Speaker Mini	45.99	36.7920
Running Shoes - Size 10	89.99	71.9920
Leather Wallet	45.00	36.0000
4K Smart TV 55 inch	600.00	480.0000
Yoga Mat	20.00	16.0000
Coffee Maker	75.50	60.4000
Smart Watch Series 5	250.00	200.0000
Cotton T-Shirt	15.00	12.0000
Gaming Mouse	50.00	40.0000
Desk Lamp	35.00	28.0000
Ceramic Mug Set	25.00	20.0000
Smart Glasses Prototype	1500.00	1200.0000
Mystery Grab Bag	15.00	12.0000

3. **Profit Margin Estimate:** Assume that for every product, our "Cost of Goods Sold" (COGS) is exactly 60% of the selling price.
   - Select product_name and price.
   - Create a calculated column for the profit per unit: price - (price * 0.60).
   - **Alias** this column as profit_per_unit.

```
-- Cost of Goods Sold (COGS) = 60% of price
-- Profit per unit = price - (price * 0.60)
SELECT product_name,
    price,
```

price - (price * 0.60) AS profit_per_unit
FROM products;

```
-- Cost of Goods Sold (COGS) = 60% of price
-- Profit per unit = price - (price * 0.60)
SELECT product_name,
       price,
       price - (price * 0.60) AS profit_per_unit
FROM products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	price	profit_per_unit
Wireless Noise-Canceling Headphones	299.99	119.9960
Ergonomic Office Chair	150.00	60.0000
Gaming Laptop Pro	1200.00	480.0000
Stainless Steel Water Bottle	25.00	10.0000
Bluetooth Speaker Mini	45.99	18.3960
Running Shoes - Size 10	89.99	35.9960
Leather Wallet	45.00	18.0000
4K Smart TV 55 inch	600.00	240.0000
Yoga Mat	20.00	8.0000
Coffee Maker	75.50	30.2000
Smart Watch Series 5	250.00	100.0000
Cotton T-Shirt	15.00	6.0000
Gaming Mouse	50.00	20.0000
Desk Lamp	35.00	14.0000
Ceramic Mug Set	25.00	10.0000
Smart Glasses Prototype	1500.00	600.0000
Mystery Grab Bag	15.00	6.0000

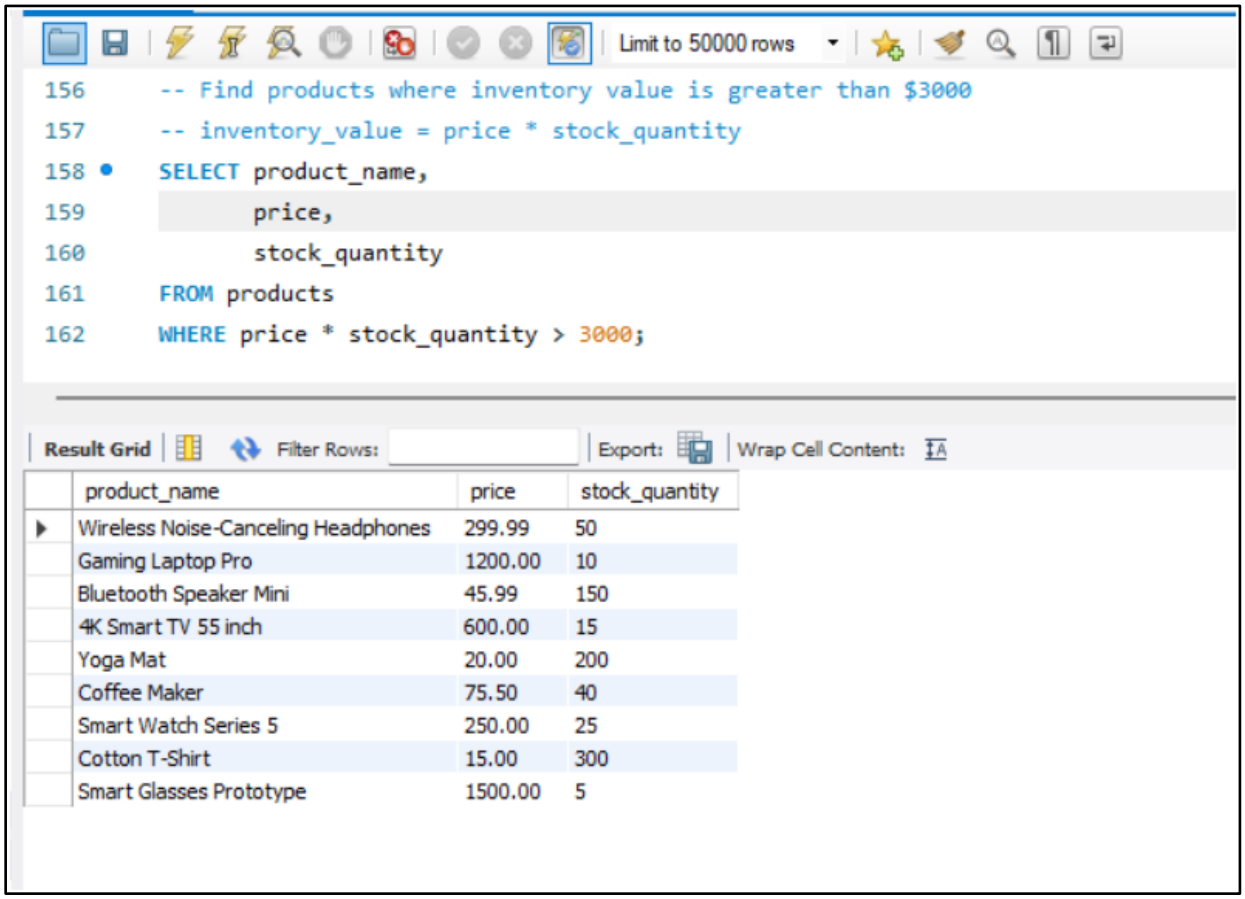
B. Arithmetic in the WHERE Clause

Pro Tip: You can filter results based on a calculation, not just raw data.

4. **High-Risk Inventory:** The warehouse manager wants to identify items where we have too much money tied up on the shelves.
   - Select product_name, price, and stock_quantity.
   - Filter to show only rows where the total inventory value (price * stock_quantity) is **greater than $3000**.

-- Find products where inventory value is greater than $3000

```
-- inventory_value = price * stock_quantity
SELECT product_name,
       price,
       stock_quantity
FROM products
WHERE price * stock_quantity > 3000;
```



product_name	price	stock_quantity
Wireless Noise-Canceling Headphones	299.99	50
Gaming Laptop Pro	1200.00	10
Bluetooth Speaker Mini	45.99	150
4K Smart TV 55 inch	600.00	15
Yoga Mat	20.00	200
Coffee Maker	75.50	40
Smart Watch Series 5	250.00	25
Cotton T-Shirt	15.00	300
Smart Glasses Prototype	1500.00	5

5. **Clearance Candidates:** We need to find cheap items that we have too many of. ○ Select product_name.
   - Filter for items where the price is **less than 25** AND the stock_quantity is **greater than 100**.

```
-- Find cheap products with too much stock

-- price < 25 and stock_quantity > 100

SELECT product_name

FROM products

WHERE price < 25
```

AND stock_quantity > 100;

```
-- Find cheap products with too much stock
-- price < 25 and stock_quantity > 100
SELECT product_name
FROM products
WHERE price < 25
AND stock_quantity > 100;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name
Yoga Mat
Cotton T-Shirt

C. Arithmetic in the ORDER BY Clause

6. **Potential Revenue Ranking:** We want to see our biggest earners at the top. ○ Select product_name and stock_quantity.
   ○ Order the results by the total inventory value (price * stock_quantity) in **Descending (DESC)** order.
   ○ *Note: In most SQL databases, you can reuse the alias from the SELECT statement in the ORDER BY clause, but standard SQL sometimes requires you to repeat the calculation.*

-- Rank products based on total inventory value

-- inventory_value = price * stock_quantity

SELECT product_name,

stock_quantity,

price * stock_quantity AS inventory_value

FROM products

ORDER BY inventory_value DESC;

```sql
-- Rank products based on total inventory value
-- inventory_value = price * stock_quantity
SELECT product_name,
       stock_quantity,
       price * stock_quantity AS inventory_value
FROM products
ORDER BY inventory_value DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	stock_quantity	inventory_value
Wireless Noise-Canceling Headphones	50	14999.50
Gaming Laptop Pro	10	12000.00
4K Smart TV 55 inch	15	9000.00
Smart Glasses Prototype	5	7500.00
Bluetooth Speaker Mini	150	6898.50
Smart Watch Series 5	25	6250.00
Cotton T-Shirt	300	4500.00
Yoga Mat	200	4000.00
Coffee Maker	40	3020.00
Ergonomic Office Chair	20	3000.00
Leather Wallet	60	2700.00
Running Shoes - Size 10	30	2699.70
Stainless Steel Water Bottle	100	2500.00
Ceramic Mug Set	80	2000.00
Mystery Grab Bag	50	750.00
Gaming Mouse	0	0.00
Desk Lamp	0	0.00

D. Complex Challenge

7. **Net Profit Projection:**
   - We want to know the total potential profit for all our 'Electronics' if we sold out everything today.
   - Select product_name.
   - Calculate the **Total Potential Profit** by taking the profit per unit (Price * 0.40) and multiplying it by stock_quantity. Alias this as projected_profit.
   - Filter only for the 'Electronics' category.
   - Order by projected_profit Descending.

-- Calculate projected total profit for Electronics category
-- Profit per unit = price * 0.40

```sql
-- projected_profit = (price * 0.40) * stock_quantity
SELECT product_name,
       (price * 0.40) * stock_quantity AS projected_profit
FROM products
WHERE category = 'Electronics'
```

SQL File 3*

Limit to 50000 rows

```sql
-- Calculate projected total profit for Electronics category
-- Profit per unit = price * 0.40
-- projected_profit = (price * 0.40) * stock_quantity
SELECT product_name,
       (price * 0.40) * stock_quantity AS projected_profit
FROM products
WHERE category = 'Electronics'
ORDER BY projected_profit DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	projected_profit
Wireless Noise-Canceling Headphones	5999.8000
Gaming Laptop Pro	4800.0000
4K Smart TV 55 inch	3600.0000
Smart Glasses Prototype	3000.0000
Bluetooth Speaker Mini	2759.4000
Smart Watch Series 5	2500.0000
Gaming Mouse	0.0000